

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.datasets import mnist
from tensorflow.keras.optimizers import Adam
```

```
print("Available GPUs:", tf.config.list_physical_devices('GPU'))
```

```
Available GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
```

```
learning_rates = [0.01, 0.001]
hidden_units = [128, 256]
```

```
batch_sizes = [32, 64]
```

```
best_accuracy = 0
best_params = {}

with tf.device('/GPU:0'):

    for lr in learning_rates:

        for units in hidden_units:

            for batch in batch_sizes:

                model = Sequential([
                    Flatten(input_shape=(28, 28)),
                    Dense(units, activation='relu'),
                    Dense(10, activation='softmax')
                ])
                model.compile(
                    optimizer=Adam(learning_rate=lr),
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy']
                )

                model.fit(
                    X_train, y_train,
                    epochs=5,
                    batch_size=batch,
                    validation_split=0.2,
                    verbose=0
                )

                _, acc = model.evaluate(X_test, y_test, verbose=0)

                print(f"LR={lr}, Units={units}, Batch={batch} → Accuracy={acc:.4f}")
                if acc > best_accuracy:
                    best_accuracy = acc
                    best_params = {
                        'learning_rate': lr,
                        'units': units,
                        'batch_size': batch
                    }
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/
super().__init__(**kwargs)
LR=0.01, Units=128, Batch=32 → Accuracy=0.9559
LR=0.01, Units=128, Batch=64 → Accuracy=0.9657
LR=0.01, Units=256, Batch=32 → Accuracy=0.9618
LR=0.01, Units=256, Batch=64 → Accuracy=0.9616
LR=0.001, Units=128, Batch=32 → Accuracy=0.9745
LR=0.001, Units=128, Batch=64 → Accuracy=0.9741
LR=0.001, Units=256, Batch=32 → Accuracy=0.9755
LR=0.001, Units=256, Batch=64 → Accuracy=0.9744
```

```
print("\nBest Accuracy from Grid Search:", best_accuracy)
print("Best Hyperparameters:", best_params)
```

Best Accuracy from Grid Search: 0.9754999876022339
 Best Hyperparameters: {'learning_rate': 0.001, 'units': 256, 'batch_size': 32}

```
final_model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(best_params['units'], activation='relu'),
    Dense(10, activation='softmax')
])
final_model.compile(
    optimizer=Adam(learning_rate=best_params['learning_rate']),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = final_model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=best_params['batch_size'],
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/10
1500/1500 ————— 6s 3ms/step - accuracy: 0.8794 - loss: 0.4207 - val_accuracy: 0.9582 - val_loss: 0.1435
Epoch 2/10
1500/1500 ————— 5s 3ms/step - accuracy: 0.9669 - loss: 0.1143 - val_accuracy: 0.9693 - val_loss: 0.1023
Epoch 3/10
1500/1500 ————— 4s 3ms/step - accuracy: 0.9791 - loss: 0.0707 - val_accuracy: 0.9712 - val_loss: 0.0913
Epoch 4/10
1500/1500 ————— 4s 3ms/step - accuracy: 0.9863 - loss: 0.0450 - val_accuracy: 0.9711 - val_loss: 0.0931
Epoch 5/10
1500/1500 ————— 5s 3ms/step - accuracy: 0.9909 - loss: 0.0322 - val_accuracy: 0.9736 - val_loss: 0.0938
Epoch 6/10
1500/1500 ————— 4s 3ms/step - accuracy: 0.9917 - loss: 0.0259 - val_accuracy: 0.9772 - val_loss: 0.0844
Epoch 7/10
1500/1500 ————— 4s 3ms/step - accuracy: 0.9944 - loss: 0.0179 - val_accuracy: 0.9767 - val_loss: 0.0823
Epoch 8/10
1500/1500 ————— 5s 3ms/step - accuracy: 0.9960 - loss: 0.0138 - val_accuracy: 0.9756 - val_loss: 0.0945
Epoch 9/10
1500/1500 ————— 4s 3ms/step - accuracy: 0.9975 - loss: 0.0096 - val_accuracy: 0.9780 - val_loss: 0.0905
Epoch 10/10
1500/1500 ————— 4s 3ms/step - accuracy: 0.9972 - loss: 0.0094 - val_accuracy: 0.9769 - val_loss: 0.0929
```

```
loss, accuracy = final_model.evaluate(X_test, y_test, verbose=0)

print(f"\nFinal Test Accuracy using Best Hyperparameters: {accuracy*100:.2f}%")
```

Final Test Accuracy using Best Hyperparameters: 98.11%